A functional interpretation of an ordered commutative rig

October 20, 2026

Abstract

This development interprets an ordered commutative unital semiring with genuine addition and multiplication function symbols in the original pure type system PTS-rk. Its domain, equality and difference order are the same certified Church construction used by the accompanying relational interpretation. The additional work concerns arbitrary nested source terms: their translations are typed, remain in the admitted domain, and respect observational equality. A separately defined functional first-order theory includes the semiring laws, strict order compatibility and all standard bounded-case axioms. Its classical deduction rules are translated directly, not by assuming a graph-elimination theorem. This note describes the mathematical model, the functional proof interface and the exact verification boundary.

1 Scope and the functional signature

The distinction from the relational development is in the source language, not in the target arithmetic. Source terms now have the grammar

$$t ::= x \mid 0 \mid S(t) \mid t + t \mid t \cdot t.$$

Thus $(x + 1) \cdot (y + z)$ is a source term. Addition and multiplication are not ternary graph predicates. They are translated to the actual closed Church addition and multiplication terms.

The primitive formulas are

$$A ::= t = t \mid t \leq t \mid \bot \mid A \to A \mid \forall x.\, A.$$

Conjunction, disjunction and existential quantification are negative abbreviations, specified in Section 2.3. There is no source predicate that asserts target typing or inhabitation.

The verified interpretation result covers 19 functional rig laws, all five functional R schemes, the strict bounded-case schema

$$\forall x.\, x < \bar{n} \iff \bigvee_{i<n} x = \bar{i} \qquad (n \in \mathbb{N}),$$

and an independent classical deduction system. The order is a strictly compatible **partial** order; arbitrary totality is not asserted. The all-cutoff quantification ranges over external natural numbers and does not mean that finitely many examples are tested.

2 The unchanged target construction

The PTS has sorts rk_i with $\mathsf{rk}_i : \mathsf{rk}_0$ and product rule

$$\frac{\Gamma \vdash A : \mathsf{rk}_a \quad \Gamma, x : A \vdash B : \mathsf{rk}_b}{\Gamma \vdash (x : A) \to B : \mathsf{rk}_{\max(a+1,b)}}.$$

All terms retain their lambda annotations. Conversion is beta conversion, without eta, cumulativity, additional primitives or changed PTS rules. The syntax and judgment are the original `PTSrk.Syntax.Term` and `Typing.typ` with `Ranks.RankSorts`.

Put $U := \mathsf{rk}_1$, so $U : \mathsf{rk}_0$, and

$$\mathsf{Void} := (Z : \mathsf{rk}_0) \to Z : \mathsf{rk}_1, \qquad \top := \mathsf{rk}_0 \to \mathsf{rk}_0 : \mathsf{rk}_1.$$

The term $\lambda(Z : \mathsf{rk}_0).Z$ inhabits $\top$. The fixed target falsity is Void, called `Bottom` in the code. No external assumption that this type is uninhabited is needed to construct the interpretation's proof terms.

2.1 Carrier and operations

The seed-first Church carrier and operations are

$$\begin{aligned} D &:= (X : U) \to X \to (X \to X) \to X : \mathsf{rk}_3, \\ 0 &:= \lambda X\, a\, f.a, \\ s\, n &:= \lambda X\, a\, f.f\,(n\, X\, a\, f), \\ \mathsf{Add}\, x\, y &:= \lambda X\, a\, f.x\, X\, (y\, X\, a\, f)\, f, \\ \mathsf{Mul}\, x\, y &:= \lambda X\, a\, f.x\, X\, a\, (\lambda u : X.y\, X\, u\, f). \end{aligned}$$

Write $1 := s\, 0$ and $\nu_n := s^n 0$. The source orientation is

$$x \oplus y := \mathsf{Add}\, y\, x, \qquad x \otimes y := \mathsf{Mul}\, y\, x.$$

The translation of a source sum is literally the closed term `Model.addition` applied to the two translated arguments. Likewise a source product applies `Model.multiplication`. The reversal above is part of their definitions; commutativity is proved and is not used to silently alter the source translation.

2.2 Goodness and fixed rank budgets

For a motive rank r, let

$$\mathsf{Good}_r(n) := (P : D \to \mathsf{rk}_r) \to P(0) \to ((u : D) \to P(u) \to P(s\, u)) \to P(n).$$

At the three budgets used here, $r = 7, 11, 15$, the certificate type has rank $r + 2$. Its zero and successor proofs are

$$g_{r,0} := \lambda P\, p_0\, p_s.p_0, \qquad g_{r,s}\, n\, g := \lambda P\, p_0\, p_s.p_s\, n\, (g\, P\, p_0\, p_s).$$

The main domain is D guarded by Good_{15}. The verified certificate-lowering maps give $\mathsf{Good}_{15} \to \mathsf{Good}_{11}$ and $\mathsf{Good}_{11} \to \mathsf{Good}_7$, not converses. Lowering explicitly raises the motive's rank using inhabited types $J_0 := \mathsf{rk}_0$ and $J_{k+1} := J_k \to \mathsf{rk}_0$; it does not invoke cumulativity.

The same three budgets are closed under zero, successor, addition and multiplication. To prove closure, open the output certificate at an arbitrary predicate, then induct into that predicate composed with the arithmetic operation. This is not induction into the goodness predicate itself. Explicit eta expansion is handled by a separate native closure proof, without eta conversion or positive certificate transport through observational equality.

These native constructions are reused from RigCodes.v and DomainClosure.v. No rank budget grows with the nesting depth of a source term.

2.3 Negative logic and observer equality

Use the proof-type encodings

$$\begin{aligned}
\neg_c P &:= P \to \mathsf{Void},\\
P \wedge_c Q &:= (P \to Q \to \mathsf{Void}) \to \mathsf{Void},\\
\bigvee_{c,i<k} P_i &:= (P_0 \to \mathsf{Void}) \to \cdots \to (P_{k-1} \to \mathsf{Void}) \to \mathsf{Void},\\
\exists_c x : A.\, P(x) &:= ((x : A) \to P(x) \to \mathsf{Void}) \to \mathsf{Void}.
\end{aligned}$$

The empty disjunction is Void. A proof type P is stable if $\neg_c\neg_c P \to P$ has a PTS inhabitant. All translated formulas are stable, but this does not assert stability of every PTS type.

For $A, B : U$, set

$$T(A,B) := (\neg_c A \to \neg_c B) \wedge_c (\neg_c B \to \neg_c A) : \mathsf{rk}_5.$$

An observer has data

$$\begin{aligned}
&X : U, \quad r : X \to X \to \mathsf{rk}_5, \quad a : X, \quad k : U \to X,\\
&f : X \to X, \quad d : X \to (U \to X) \to X, \quad p : X \to U,
\end{aligned}$$

and nine certificates:

$$\begin{aligned}
&r(v,v),\\
&r(v,w) \to r(w,v),\\
&r(u,v) \to r(v,w) \to r(u,w),\\
&r(v,w) \to r(fv,fw),\\
&((z : U) \to r(hz,h'z)) \to r(d_w h, d_w h'),\\
&r(f(kz),kz),\\
&r(d_w k, w),\\
&r(d_w(\lambda z : U.v), v),\\
&r(v,w) \to T(pv,pw).
\end{aligned}$$

Here $d_w h := d\,w\,h$; the variables $u, v, w : X$, $z : U$, $h, h' : U \to X$ are universally bound in their respective laws. All nine proofs are fields of the observer telescope, not assumptions added to the PTS.

If $\mathcal{A}$ denotes this complete dependent telescope, define

$$\mathsf{E}(x,y) := \mathcal{A} \to T\big(p(x\,X\,a\,f), p(y\,X\,a\,f)\big) : \mathsf{rk}_7.$$

The telescope notation expands dependent arrows and introduces no quotient or record primitive. This equality is stable and an equivalence. It respects successor, explicit eta expansion, and addition/multiplication on appropriately certified arguments. It is not identified with beta conversion, nor is every raw function $D \to D$ assumed to preserve it.

2.4 Successor separation

The two successor facts require actual observers, not an assumption that the observer telescope separates Church terms. For nonzeroness, take the concrete data

$$\begin{aligned} X_0 &:= U \to U \to U, \\ r_0(v,w) &:= (A, B : U) \to T(v\,A\,B, w\,A\,B), \\ a_0\,A\,B &:= A, \qquad\qquad k_0\,C\,A\,B := C, \\ f_0\,v\,A\,B &:= v\,B\,B, \\ d_0\,w\,h\,A\,B &:= h(w\,A\,B)\,A\,B, \\ p_0\,v &:= v\,\top\,\mathsf{Void}. \end{aligned}$$

Pointwise reflexivity, symmetry and composition of T supply its nine certificates. Zero is observed as $\top$; induction with $\mathsf{Good}_7(x)$ gives T between the successor observation and Void. An assumed $\mathsf{E}(s\,x, 0)$ would therefore relate Void and $\top$ via T. Its negative implications, the identity on Void and the closed inhabitant of $\top$ yield a term of Void.

For injectivity, start with any observer and build its delayed version:

$$\begin{aligned} X' &:= U \to X, \\ r'(v,w) &:= (z : U) \to r(vz, wz), \\ a' &:= k, \qquad\qquad k'\,z := \lambda u : U.k\,z, \\ f'\,v &:= \lambda u : U.d\,a\,(\lambda z : U.f(vz)), \\ d'\,w\,h &:= \lambda u : U.d(wu)(\lambda z : U.h\,z\,u), \\ p'\,v &:= p(v\,\top). \end{aligned}$$

The original certificates prove all the new ones. In particular, the state parameter of d is retained; eta-expanded functions are compared by its pointwise-congruence law, not converted by an eta rule. The delayed carrier still has rank one.

Write $F_n := n\,X\,a\,f$ and $F'_n := n\,X'\,a'\,f'$. Induction with $\mathsf{Good}_7(n)$, using an explicit lift of the rank-five simulation motive, proves

$$(z : U) \to r(F'_{s\,n}z, F_n).$$

At zero, use $f(kz) \sim kz$ and $d_a k \sim a$; at successor, use pointwise congruence and the constant-function law for d_a. Apply an assumed $\mathsf{E}(s\,x, s\,y)$ to this delayed observer, and compose its result with the two simulations through the p bridge. For $\mathsf{Good}_7(x)$ and $\mathsf{Good}_7(y)$, this recovers the result of every original observer, hence $\mathsf{E}(x,y)$. Both constructions are native terms in SuccessorSeparation.v.

2.5 Order and the reused arithmetic theorems

The difference order is

$$\mathsf{O}(x,y) := ((c : D) \to \mathsf{Good}_{11}(c) \to \mathsf{E}(\mathsf{Add}\,c\,x, y) \to \mathsf{Void}) \to \mathsf{Void} : \mathsf{rk}_{15}.$$

Its witnesses satisfy $x \oplus c =_E y$. They carry Good_{11} even when the endpoints carry Good_{15}. Define $x < y := \mathsf{O}(x,y) \wedge_c \neg_c \mathsf{E}(x,y)$, of rank seventeen.

The accompanying relational construction already supplies the following native proof terms for this same carrier and equality:

- Successor injectivity and nonzeroness with Good_7 guards, using concrete separating and delayed observers with all nine certificates explicitly constructed.
- Commutative unital semiring equations, additive cancellation, zero-sum reflection and the negative zero-product disjunction.
- Reflexivity, transitivity, antisymmetry, zero leastness, and operation monotonicity for the difference order.
- Strict addition and multiplication laws, the latter requiring a positive factor.

The algebraic inductions use a continuation invariant retaining lower goodness evidence: for stable P of rank seven, eight or nine, $I(u) := \mathsf{Good}_7(u) \wedge_c P(u)$ has rank eleven. Induction with Good_{11} proves $I(x)$, from which stability yields $P(x)$. The lower certificate is used only inside the continuation, not extracted positively.

For strict multiplication, a hypothetical equality of products would make the product of a difference and the positive factor zero. The zero-product disjunction then contradicts either the original strict inequality or positivity of the factor. This argument is stronger than merely proving monotonicity.

Every cutoff is handled by the same rank-fifteen flattened finite-case invariant

$$\begin{aligned} K_n(x) &:= (t : D) \to \mathsf{Good}_{11}(t) \to \mathsf{E}(x, s^{n+1}t) \to \mathsf{Void}, \\ \mathsf{Cut}_n(x) &:= (\mathsf{E}(x, \nu_0) \to \mathsf{Void}) \to \cdots \to (\mathsf{E}(x, \nu_n) \to \mathsf{Void}) \to K_n(x) \to \mathsf{Void}. \end{aligned}$$

The finite telescope grows with n, not its rank. Induction on a Good_{15} input shifts finite branches, sends the last finite branch to the large branch with witness zero, and increments the witness in the large branch. Successor separation rules out the large case under a contrary finite bound. This proves the inclusive finite-downset equivalence, comparison with each numeral, and the strict finite-downset equivalence, including zero. No same-domain predecessor or promotion of Good_{11} to Good_{15} is used.

An expanded account of the common arithmetic proof terms is available in the relational note as optional further reading. The functional proof obligations are described next.

3 Arbitrarily nested source terms

Let ρ assign target objects to source variables. The translation is defined structurally:

$$\begin{aligned} x^*_\rho &:= \rho(x), & 0^*_\rho &:= 0, \\ S(t)^*_\rho &:= s(t^*_\rho), \\ (t+u)^*_\rho &:= t^*_\rho \oplus u^*_\rho, \\ (t \cdot u)^*_\rho &:= t^*_\rho \otimes u^*_\rho. \end{aligned}$$

There is no existentially chosen output for an arithmetic subterm. Its translation is an actual application of the target operation.

Lemma 3.1 (Term typing and domain closure)**.** *If every $\rho(x)$ has type D, then every t^*_ρ has type D. If, in addition, every $\rho(x)$ carries a Good_{15} certificate, then every t^*_ρ carries such a certificate.*

The proof follows the five source-term constructors. The two new binary cases apply native operation typing and the concrete goodness-closure proofs to the induction hypotheses. They do not require a larger goodness index or arbitrary induction on the target carrier.

Lemma 3.2 (Term congruence)**.** *Suppose two source environments consist of typed* Good_{15} *objects and satisfy* $\mathsf{E}(\rho(x), \sigma(x))$ *at every variable. Then* $\mathsf{E}(t^*_\rho, t^*_\sigma)$ *for every source term.*

At a sum or product, first obtain the translated subterms' certificates by Lemma 3.1. Lower them to the budgets needed by the specific arithmetic congruence lemmas. Then use the induction hypotheses for both arguments. The new term-congruence induction is essential: the relational signature had no addition or multiplication constructors inside its source terms.

Translation.v also proves compatibility with renaming and substitution, including the source-variable lifting under binders. Equality.v uses the term-congruence theorem to prove formula transport and equality replacement for arbitrary functional formulas.

4 The independent functional theory

Source.v defines `FunctionalSource`. Its 19 universally closed rig axioms are the functional forms of the relational model's 23 laws, with the four graph totality and uniqueness axioms removed:

1. Addition and multiplication commutativity and associativity (four laws).
2. Additive zero, multiplicative zero and unit, distributivity and additive cancellation (five laws).
3. Order reflexivity, transitivity and antisymmetry, zero leastness and $0 \neq 1$ (five laws).
4. Monotonicity of both operations, strict additive monotonicity and strict multiplicative monotonicity with a positive factor (four laws).
5. The successor equation $x + 1 = S(x)$.

For instance, associativity is the direct equation $(x+y)+z = x+(y+z)$. The target's closed operation terms, their typing and Lemma 3.1 supply the total functional interpretation; graph witnesses are neither source axioms nor hidden premises.

The original five functional R schemes are

$$\begin{aligned}
&\bar{m} + \bar{n} = \overline{m+n},\\
&\bar{m} \cdot \bar{n} = \overline{mn},\\
&\bar{m} \neq \bar{n} \quad (m \neq n),\\
&\forall x.\ x \leq \bar{n} \to \bigvee_{i \leq n} x = \bar{i},\\
&\forall x.\ x \leq \bar{n} \vee \bar{n} \leq x.
\end{aligned}$$

Here m, n are arbitrary external natural numbers. The fourth original scheme is an implication. The model additionally proves the stronger inclusive equivalence and the strict equivalence from Section 1. The strict schema is a separate source axiom constructor for every n, including $n = 0$.

Negation means implication to falsity; conjunction and finite disjunction are the negative abbreviations of Section 2.3; the source existential abbreviation is $\neg\forall x \neg A$. The independently defined deduction predicate has exactly nine constructors:

1. An assumption from the source context.
2. An instance of a source theory axiom.
3. Implication introduction.

4. Implication elimination.
5. Universal introduction with renamed context.
6. Universal elimination by a source term.
7. Classical double-negation elimination.
8. Equality reflexivity.
9. Equality replacement in any source formula.

The source syntax shares finite indices with the native library, but no derivability rule mentions target typing, semantic validity or the theorem to be proved.

5 Translation of formulas and derivations

The atomic and logical clauses are

$$\begin{aligned}(t = u)^*_\rho &:= \mathsf{E}(t^*_\rho, u^*_\rho),\\ (t \le u)^*_\rho &:= \mathsf{O}(t^*_\rho, u^*_\rho),\\ \bot^*_\rho &:= \mathsf{Void},\\ (A \to B)^*_\rho &:= A^*_\rho \to B^*_\rho,\\ (\forall x.\, A)^*_\rho &:= (x : D) \to \mathsf{Good}_{15}(x) \to A^*_{\rho[x]}.\end{aligned}$$

Formula typing computes a finite rank structurally: equality has rank seven, order fifteen, falsity one, implication has rank $\max(a + 1, b)$, and universal quantification has rank $\max(18, b)$. Arbitrary formulas are not all bounded by rank eighteen: implication nesting can raise the result. This does not change the fixed carrier, equality, goodness or finite-cut budgets used in the model.

Stability of the translated atoms and closure of stability under the logical clauses give double-negation elimination for every translated source formula. Equality replacement follows by structural formula transport, using Lemma 3.2 for both kinds of atom. Neither step identifies observational equality with target conversion.

The generic theorem in Deduction.v takes an explicit axiom-realization argument and performs induction on the nine source rules. Universal elimination is the important functional case: the instantiated term may contain arbitrary nested addition and multiplication. Its actual target term and goodness certificate are supplied by the structural term theorems, and the substitution lemma identifies the resulting formula.

AxiomRealization.v supplies the actual translated rig and R axioms from the already proved native arithmetic, order and bounded-case inhabitants. The public theorem in TheoremFunctionalSemiring.v discharges the generic axiom-realization argument. Its unconditional type is

```
theorem_functional_semiring :
  (forall A, FunctionalSource.axiom A ->
    exists p, @typ nat RankSorts 0 ε p
      (FunctionalTranslation.sentence A)) /\
  (forall A, FunctionalSource.derives [] A ->
    exists p, @typ nat RankSorts 0 ε p
      (FunctionalTranslation.sentence A)).
```

The first premise is membership in an independent source axiom schema; the second is an independent source derivation. Neither assumes a target inhabitant. All conclusions use the original native typing judgment, not checker acceptance.

5.1 Ground evaluation and internal refutation

A closed functional source term now evaluates recursively through both arithmetic constructors:

$$|t + u| = |t| + |u|, \qquad |t \cdot u| = |t| \cdot |u|.$$

GroundFaithfulness.v relates the literal target translation to $\nu_{|t|}$ by beta conversion. For a true numerical equality or order test, it constructs the corresponding target proof; for a false test, it constructs an internal refutation mapping that atom to Void. Thus the test applies to all closed nested terms, not merely displayed numerals. The four uniform clauses are combined in `FunctionalGround.ground_faithfulness`.

These internal refutations do not alone prove external non-inhabitation of Void. No set model or external consistency theorem is assumed in the construction, and Rocq's native arithmetic is used only to justify side conditions and select the actual target proof or refutation inhabitants.

6 Boundaries and reproduction

Nothing in the functional signature promotes Good_{11} differences to Good_{15}. The cutoff-zero case therefore does not give a predecessor inside the main domain, and no interpretation of full Robinson Q is claimed here. Likewise comparison with each standard numeral is not arbitrary totality, trichotomy or strict cotransitivity.

The source algebra and all finite-downset schemes themselves do not force linearity. For example, let C be zero together with the integer polynomials eventually positive at both $+\infty$ and $-\infty$, with $p \leq q$ iff $q - p \in C$. It is a cancellative ordered commutative rig with strict multiplication and the correct downset of every standard numeral. Yet $t^2 + t$ and $t^2 - t$ are incomparable. Taking $x = t^2$, $y = t^2 + t$ and $z = t^2 + 1$ also refutes strict cotransitivity. This source counterexample is mathematical analysis, not a new Rocq theorem or a proof that the particular PTS interpretation cannot satisfy a stronger comparison principle.

The library `OrderedSemiringFunctional` is separate from `OrderedSemiring` and reuses its native mathematical infrastructure. There is no dependency on the repository's other functional R interpretation, set model, or additional PTS primitives.

The code map is compact:

- Source.v: functional syntax, substitution, independent axioms and rules.
- Translation.v: literal term translation, typing, goodness, formula translation, stability and substitution.
- Equality.v: nested-term congruence, formula transport and replacement.
- AxiomRealization.v: actual source-axiom realizers.
- Deduction.v: induction on independent source deductions.
- GroundFaithfulness.v: nested-term evaluation and ground internal proofs and refutations.
- TheoremFunctionalSemiring.v: the unconditional public interpretation.

On 20 October 2026, a clean Git archive of proof revision 9cf6a19 was built in a fresh absolute Dune directory. Compilation and the subsequent recursive `rocq check` both exited successfully. The audit includes both ordered-semiring interpretations and their shared dependencies, without relying on loose working-tree files.

The final interpretation, strict bounded-case, numeral-comparison and ground-faithfulness theorems all have closed `Print Assumptions` reports. The complete fresh build emitted 137 closed reports and no axiom dependencies. There are no admitted proofs, new axioms, disabled safety checks or target-theorem hypotheses. The environment was Rocq 9.2.0, Stdlib 9.1.0, Equations 1.3.2+9.2, OCaml 4.14.2 and Dune 3.23.1.

From the repository root, the native audit is

```
bash theories/OrderedSemiringFunctional/check.sh
```

The driver records versions, exact commands and exit codes, regenerates assumption output in a fresh build, and recursively kernel-checks the public theorems and dependencies. Set `ORDERED_FUNCTIONAL_LOG_DIR` to choose the evidence directory. It performs no package installation or network access.

To build this documentation from the repository root, run

```
bash theories/OrderedSemiringFunctional/doc/build.sh
```

The build uses the same bundled Markdown class, plugin, fonts and mathematical typography as the relational note. It checks the shared runtime's hashes, snapshots all inputs into a fresh temporary directory, disables shell escape and performs three LuaLaTeX passes. The rendered-source audit rejects literal environment fences, escaped mathematics and empty display blocks; final-pass diagnostics reject missing glyphs and unresolved references. No second copy of the Markdown runtime is vendored, and no network, bibliography tool or external syntax highlighter is required. The resulting local document is README.pdf.